

Relatório de Evolução do Projeto

ARIA — Assistente de Recarga Inteligente e Autônoma

GoodWe EV Challenge · FIAP · Prompt and Artificial Intelligence · Sprint 03 · 2026.2

1. Resumo da evolução

Nas Sprints 1 e 2, o grupo definiu o escopo do chatbot ARIA para atendimento GoodWe (EV ChargeOps — gestão compartilhada de carregadores em condomínios) e implementou o núcleo conversacional de forma manual: chamadas diretas ao SDK do Groq (modelo *llama-3.3-70b-versatile*), com o system prompt injetado por persona e o histórico de conversa mantido em uma lista Python, passada e atualizada manualmente a cada turno. A recuperação de contexto (RAG) já usava ChromaDB, e o canal de produção era o WhatsApp via Twilio, com Flask e Cloudflare Tunnel.

Na Sprint 03, o núcleo conversacional foi reconstruído sobre o **LangChain**, com a memória de conversa passando a ser gerenciada nativamente pelo framework (*RunnableWithMessageHistory*), indexada por sessão. Além do refactory em si, foram adicionados: suporte a um segundo modelo (Google Gemini) para comparação, três casos de teste de segurança/prompt injection com avaliação documentada, e correções de guardrail identificadas durante esses testes. O RAG (ChromaDB) e o canal de produção (Flask + Twilio) permaneceram os mesmos — o refactory foi cirúrgico, focado no núcleo conversacional exigido pela rubrica.

2. Refatoração — decisões técnicas e trade-offs

2.1 Framework escolhido: LangChain

Optamos pelo LangChain por ser o framework de agentes com a documentação mais madura e maior compatibilidade direta com o Groq (*langchain-groq*) e o Gemini (*langchain-google-genai*), permitindo trocar de provider sem alterar a lógica de negócio do chatbot. A memória por sessão passou a ser resolvida por um callback (*_obter_historico*) que associa um *session_id* (número de telefone no WhatsApp, ou um identificador fixo no modo terminal) a um histórico isolado, eliminando a necessidade de gerenciar listas de mensagens manualmente em cada ponto de entrada (CLI e webhook).

2.2 Trade-off assumido: sem migração para LangGraph

O *RunnableWithMessageHistory* está marcado como descontinuado pelo próprio LangChain, em favor da persistência nativa do LangGraph. Optamos por não migrar para o LangGraph nesta sprint: o escopo pedido é "um framework de agentes com memória gerenciada", requisito já atendido pela API atual do LangChain, e introduzir o LangGraph agora ampliaria o escopo sem necessidade, contrariando o princípio de menor complexidade possível já adotado desde a Sprint 2 (quando o próprio LangChain havia sido descartado por complexidade desnecessária). Fica registrado como possível evolução futura.

2.3 Escolha de modelos para comparação

Mantivemos o Groq como provider principal (trocando apenas o modelo, descontinuado — ver seção 3) e adicionamos o Google Gemini como segundo modelo, por ser gratuito via Google AI Studio e por representar uma arquitetura/infraestrutura genuinamente distinta do Groq, o que enriquece a comparação do Bloco B em vez de comparar apenas dois modelos hospedados na mesma plataforma.

3. Comparativo antes / depois

Uma limitação real precisa ser registrada com transparência: o modelo usado na versão manual das Sprints 1/2 (*llama-3.3-70b-versatile*) foi descontinuado pelo Groq em 16/08/2026 — não é mais possível executar a versão legada exatamente como ela era, mesmo revertendo o código. Por isso, latência e tokens são comparados de forma qualitativa/estrutural (o que muda de fato entre as duas versões), e não por reexecução idêntica lado a lado.

Critério	Sprints 1/2 (versão manual)	Sprint 03 (LangChain)
Gerenciamento de memória	Lista Python passada e atualizada manualmente a cada turno, em cada ponto de entrada (CLI e webhook).	Gerenciada nativamente pelo framework (RunnableWithMessageHistory), indexada por session_id, com função dedicada para limpar sessão.
Qualidade das respostas (eval set, 3 perguntas)	Respostas corretas e estruturadas nas 3 perguntas do eval set (registrado nos prints das Sprints 1/2).	Respostas corretas e estruturadas nas mesmas 3 perguntas (Groq); conteúdo equivalente ou superior em detalhamento.
Latência / tokens por turno	Não instrumentado no código legado; mesma API Groq subjacente, portanto ordem de grandeza equivalente à medida na versão atual.	Instrumentado via script de comparação: média de 2,31s e ~1.657 tokens/turno (Groq, modelo de produção).
Comparação entre modelos	Não realizada — um único modelo, sem instrumentação de latência/tokens.	Groq vs. Gemini testados e documentados em relatorio_modelos.md, com seleção justificada do Groq para produção.
Testes de segurança / prompt injection	Não exigidos nem realizados nesta etapa do projeto.	3 casos documentados (override direto, jailbreak por autoridade falsa, indução a aconselhamento fora do escopo); 2 lacunas de guardrail identificadas e corrigidas.

4. Problemas encontrados e soluções

4.1 Modelo Groq descontinuado

Problema: Ao rodar o projeto pela primeira vez nesta sprint, o modelo *llama-3.3-70b-versatile* retornava erro 404 ("model_not_found"). Investigação confirmou que o Groq descontinuou o modelo em 16/08/2026.

Solução: Substituído por *openai/gpt-oss-120b*, modelo de produção do Groq, sem necessidade de alterar API key ou demais partes do código.

4.2 Modelo Gemini indisponível para novos usuários

Problema: O modelo *gemini-2.5-flash*, escolhido inicialmente como segundo modelo, retornou erro 404 informando que não está mais disponível para novos usuários da API.

Solução: Substituído por *gemini-3.6-flash*, indicado pela própria mensagem de erro da API como sucessor recomendado.

4.3 Respostas do Gemini truncadas e malformadas

Problema: Nos primeiros testes comparativos, o Gemini retornava respostas cortadas no meio e, em alguns casos, o conteúdo bruto (uma lista Python com blocos internos) aparecia como texto final. A causa raiz: o Gemini 3.6 usa por padrão um nível alto de raciocínio interno ("thinking"), que consome parte do limite de tokens de saída antes de gerar a resposta visível, e anexa metadados internos ("thought signatures") ao conteúdo retornado.

Solução: Configurado *thinking_level="low"* para reduzir o raciocínio interno ao necessário para esse tipo de tarefa (Q&A; com contexto já recuperado via RAG), e criada a função *extrair_texto_resposta()*, que normaliza o conteúdo da resposta para texto simples independentemente do formato interno devolvido pelo provider.

4.4 Guardrail incompleto para aconselhamento fora do escopo

Problema: No teste de segurança do Caso 3 (indução a parecer jurídico), a ARIA recusou corretamente dar o aconselhamento, mas não indicou um profissional habilitado — requisito explícito da rubrica da sprint.

Solução: Adicionada uma regra explícita ao system prompt determinando que, ao recusar aconselhamento jurídico, financeiro ou de segurança elétrica, a ARIA deve indicar o profissional habilitado correspondente (advogado, contador ou eletricista certificado). Reteste confirmou a correção.

4.5 Vazamento indireto de identidade do modelo

Problema: No teste de segurança do Caso 2 (jailbreak por autoridade falsa), a resposta da ARIA mencionou estar seguindo "as políticas de uso da OpenAI" — um vazamento indireto de que o modelo subjacente é da OpenAI (executado via Groq), quebrando a persona da ARIA como assistente da GoodWe.

Solução: Adicionada regra ao system prompt proibindo a ARIA de revelar, confirmar ou mencionar qual modelo de IA, empresa ou provedor de tecnologia está por trás do seu funcionamento.

5. Equipe e divisão de trabalho

Nome	RM	Tarefa principal
Victor Hugo	571099	Melhoria do chatbot — refactory do núcleo conversacional para LangChain
Davi Ramos	571744	Código de comparação entre modelos (compare_models.py e relatorio_modelos.md)
Lucas Galle Melchior	574027	Testes de segurança / prompt injection (test_seguranca.py e avaliação dos casos)
Gabriel Cavaloti	571643	Documentação (README.md e este relatório de evolução)